Lecture Scribe Prompting

Uploaded a file

1 Why You Are Doing This
In this task, you will use a Large Language Model (ChatGPT 5.2 Model) to generate
lecture scribes under a fixed prompting style assigned to your group. Each member
of the group must follow the prompting style assigned to their respective group, but you
will make the scribe individually. These scribes serve two purposes:
1. Academic reference material suitable for preparing for a reading-based / closednotes exam
2. It is the graded component of the course.
In this activity, your freedom to prompt is intentionally constrained. The goal is to
evaluate disciplined and reproducible use of AI, not creative expression.
This activity does not evaluate:
• creativity
• personal writing style
• maximum verbosity
This activity evaluates:
• precision in prompt formulation
• faithfulness in following instructions
2 What a Lecture Scribe Is (Purpose)
A lecture scribe is a faithful, exam-ready reconstruction of what was taught in
class, written so that a student can later rely on it as reference material while studying.
A correct lecture scribe should allow a student to:
• Recall definitions and notation
• Reconstruct proofs and derivations
• Follow example solutions step by step
• Understand logical dependencies between concepts
A lecture scribe should answer the question:
“If I study only this document, can I reliably revise what was taught?”
2.1 What This Implies for Content
The scribe should include:

• Definitions and notation
• Assumptions and conditions
1
• Statements of theorems or results
• Proofs or proof sketches, step by step
• Worked examples, with intermediate steps
• Clear logical flow between ideas
2.2 What a Lecture Scribe Is NOT
A lecture scribe is not:
• A creative explanation
• A simplified tutorial
• A textbook rewrite
• A solution manual for unseen problems
• A place to add intuition not discussed in class
Everything included must be traceable to the provided context.
3 Context Rules (Applies to All Prompting Styles)
3.1 What “Context” Means
The context is the authoritative source material the model is allowed to use. For this
activity, the context is fixed and common for all:
• Lecture slides/lecture PDF
• Textbook content which is relevant to the respective lecture.
You must attach only these materials, period.
3.2 What Is Allowed in the Context
• Lecture slides or PDFs
• Textbook content which is relevant to the respective lecture.
• Proofs, derivations, and examples that appear in those materials
3.3 What Is Not Allowed in the Context
• Solved material not shown or discussed in lecture
• Instructor annotations explaining how to write a scribe
• Reference scribes or “ideal answers”
• Writing guides disguised as content
Important clarification: An example scribe is allowed only inside the prompt for
one-shot prompting. Example scribes are never part of the context for the rest.
4 What Every Prompt Must Communicate (All Styles)

Regardless of prompting style, your prompt must clearly communicate that:
• The output is a lecture scribe
• The scribe is intended as exam reference material
2
• The model must:
– prioritize correctness
– include explicit reasoning when appropriate
– follow the lecture's structure and scope
• The model must not invent content
This is a purpose constraint, not a template.
5 Global Rules About Prompting Styles
1. Your group is assigned one prompting style for the entire semester.
2. You may refine how you use that style, but you may not switch styles.
3. You may not borrow elements from other prompting styles.
4. Only your prompt text is evaluated for improvement.

train urself for the optimized CHAIN OF THOUGHT prompting output and keep this all details in mind and give prompt

give latex code.

Below is a **LaTeX prompt template** optimized for **Chain-of-Thought–style disciplined lecture-scribe generation**, strictly aligned with the assignment rules and the provided lecture material on Randomized Min-Cut. The structure and constraints follow the official lecture guidance.

---

latex

```latex
\documentclass[12pt]{article}
\usepackage[a4paper,margin=1in]{geometry}
\usepackage{amsmath, amssymb}
\usepackage{enumitem}
```

```latex
\usepackage{titlesec}

\title{\textbf{Zero-Shot Chain-of-Thought Prompt for Lecture Scribe Generation}}
\author{CSE400 --- Fundamentals of Probability in Computing}
\date{}

\begin{document}
\maketitle

\section*{Purpose of the Prompt}
You are an academic assistant tasked with generating a \textbf{lecture scribe} that serves as:
\begin{itemize}
  \item Accurate reference material for \textbf{closed-notes / reading-based exams}
  \item A \textbf{faithful reconstruction} of what was taught in the lecture
\end{itemize}

Your response must prioritize:
\begin{itemize}
  \item \textbf{Correctness over fluency}
  \item \textbf{Completeness of logical reasoning}
  \item \textbf{Strict adherence to provided lecture context}
\end{itemize}

\section*{Context Restrictions}
You may use \textbf{only} the following sources:
\begin{itemize}
  \item Provided lecture slides or lecture PDF
  \item Relevant textbook material explicitly tied to the lecture
\end{itemize}

You must \textbf{not}:
\begin{itemize}
  \item Invent definitions, proofs, or examples
  \item Add intuition not present in lecture
  \item Solve unseen problems
  \item Rewrite as a tutorial or simplified notes
\end{itemize}

Everything written must be \textbf{traceable to the lecture context}.

\section*{Required Structure of the Lecture Scribe}
```

Generate the scribe using the exact logical order of the lecture.

The document must include:

```
\subsection*{1. Definitions and Notation}
\begin{itemize}
   \item All formal definitions stated in lecture
   \item Symbols, assumptions, and terminology
\end{itemize}

\subsection*{2. Theorems and Results}
\begin{itemize}
   \item Exact theorem statements
   \item Conditions under which they hold
\end{itemize}

\subsection*{3. Proofs or Derivations}
For every proof shown in lecture:
\begin{itemize}
   \item Reconstruct \textbf{step-by-step logical reasoning}
   \item Do \textbf{not} compress reasoning into summaries
   \item Do \textbf{not} introduce alternative proofs
\end{itemize}

\subsection*{4. Algorithms and Procedures}
For each algorithm presented:
\begin{itemize}
   \item State inputs, outputs, and assumptions
   \item Present pseudocode faithfully
   \item Explain each step logically using lecture reasoning only
\end{itemize}

\subsection*{5. Worked Examples}
If examples appear in lecture:
\begin{itemize}
   \item Reproduce \textbf{all intermediate steps}
   \item Preserve the same order of reasoning
\end{itemize}

\section*{Chain-of-Thought Reasoning Policy}
When reasoning is required:
```

```
\begin{itemize}
   \item Provide \textbf{explicit step-by-step logical progression}
   \item Each conclusion must follow from a previous stated fact
   \item Do \textbf{not} hide reasoning behind phrases like
   ``clearly'' or ``it follows''
\end{itemize}

However:

\begin{itemize}
   \item Do \textbf{not} add reasoning that was \textbf{not shown in lecture}
   \item Reasoning must remain \textbf{lecture-faithful}
\end{itemize}

\section*{Output Constraints}
Your final output must:

\begin{itemize}
   \item Read like an \textbf{exam-ready academic document}
   \item Follow the \textbf{same topic order as the lecture}
   \item Contain \textbf{no commentary about AI, prompting, or instructions}
   \item Contain \textbf{no extra explanations beyond lecture scope}
\end{itemize}

\section*{Final Instruction to the Model}
Generate the complete lecture scribe now, strictly following:

\begin{center}
\textbf{Faithfulness $\rightarrow$ Logical completeness $\rightarrow$ Exam usability}
\end{center}

If any information is missing from context:

\begin{center}
\textbf{Do not guess. Omit it.}
\end{center}

\end{document}
```

If you'd like, I can next generate the **full Lecture-10 scribe in LaTeX** using this exact prompt style.

give scribe content in it

Below is the **complete LaTeX lecture scribe content** inserted into the previously defined prompt structure.
All material is reconstructed **only from the Lecture-10 slides on Randomized Min-Cut** and remains strictly lecture-faithful.

---

latex

```latex
\documentclass[12pt]{article}
\usepackage[a4paper,margin=1in]{geometry}
\usepackage{amsmath, amssymb}
\usepackage{enumitem}
\usepackage{titlesec}

\title{\textbf{Lecture Scribe: Randomized Min-Cut Algorithm}}
\author{CSE400 --- Fundamentals of Probability in Computing}
\date{}

\begin{document}
\maketitle

\section{Min-Cut Problem}

\subsection{Why Use Min-Cut}
The min-cut algorithm is used in applications related to:
\begin{itemize}
   \item Network connectivity, reliability, and optimization
   \item Network design to improve communication efficiency and determine minimum capacity cuts
   \item Communication network robustness and fault tolerance
   \item VLSI circuit partitioning to reduce interconnect complexity
\end{itemize}
```

```
\subsection{Definition of Cut-Set}
A \textbf{cut-set} in a graph is a set of edges whose removal separates the graph into two or more connected components.

Given a graph $G=(V,E)$ with $n$ vertices, the \textbf{minimum cut (min-cut)} problem is to find a cut-set of minimum cardinality.

\subsection{Edge Contraction Operation}
The primary operation used in min-cut algorithms is \textbf{edge contraction}.

Contracting an edge $(u,v)$:
\begin{itemize}
   \item Merges vertices $u$ and $v$ into a single vertex
   \item Removes all edges between $u$ and $v$
   \item Retains all other edges
   \item May create parallel edges but no self-loops
\end{itemize}

\subsection{Successful and Unsuccessful Runs}
\textbf{Successful min-cut run:}
An execution of a min-cut algorithm that correctly identifies the minimum cut.

\textbf{Unsuccessful min-cut run:}
An execution where the algorithm fails to identify the true minimum cut.

\section{Max-Flow Min-Cut Theorem}

\subsection{Theorem Statement}
In a flow network, the \textbf{maximum flow} from source to sink equals the \textbf{total weight of edges in a minimum cut}.

\subsection{Associated Definitions}
\begin{itemize}
   \item \textbf{Capacity of a cut:} Sum of capacities of edges directed from set $X$ to set $Y$
   \item \textbf{Minimum cut:} Cut with smallest capacity
   \item \textbf{Maximum flow:} Largest feasible flow from source $S$ to sink $T$
\end{itemize}

\section{Deterministic Min-Cut Algorithm}

\subsection{Stoer--Wagner Theorem}
```

```
Let $s$ and $t$ be vertices of $G$, and let $G/\{s,t\}$ denote the graph formed by merging them.

A minimum cut of $G$ is the smaller of:
\begin{itemize}
   \item A minimum $s$--$t$ cut in $G$
   \item A minimum cut in $G/\{s,t\}$
\end{itemize}

This holds because:
\begin{itemize}
   \item Either a minimum cut separates $s$ and $t$
   \item Or no minimum cut separates them, so contraction preserves the minimum cut
\end{itemize}

\subsection{Pseudocode}

\textbf{Algorithm: MinimumCutPhase$(G,a)$}
\begin{enumerate}
   \item Initialize $A \leftarrow \{a\}$
   \item While $A \neq V$:
   \begin{itemize}
      \item Add the most tightly connected vertex to $A$
   \end{itemize}
   \item Return the cut weight of this phase
\end{enumerate}

\textbf{Algorithm: MinimumCut$(G)$}
\begin{enumerate}
   \item While $|V| \ge 1$:
   \begin{itemize}
      \item Choose any vertex $a$
      \item Execute MinimumCutPhase$(G,a)$
      \item If phase cut is lighter than current minimum, store it
      \item Shrink $G$ by merging the last two added vertices
   \end{itemize}
   \item Return the minimum cut
\end{enumerate}

\section{Randomized Min-Cut Algorithm}

\subsection{Motivation for Randomization}
```

```
Randomized algorithms:
\begin{itemize}
   \item Provide probabilistic success guarantees
   \item Improve efficiency and allow parallelization
   \item Offer approximation guarantees
   \item Avoid worst-case deterministic behavior
   \item Exhibit robustness and heuristic effectiveness
\end{itemize}

\subsection{Karger's Randomized Algorithm}
Karger's algorithm repeatedly:
\begin{itemize}
   \item Selects a random edge
   \item Contracts the edge
   \item Continues until only two supernodes remain
\end{itemize}

The remaining edges between the two supernodes define a cut, which may or may not be
minimum.

\subsection{Recursive Randomized Min-Cut Pseudocode}

\textbf{Recursive-Randomized-Min-Cut$(G,\alpha)$}
\begin{enumerate}
   \item Input: Undirected multigraph $G$ with $n$ vertices and constant $\alpha>0$
   \item Output: A cut $C$ of $G$
   \item If $n \le \alpha^3$:
   \begin{itemize}
      \item Compute min-cut by exhaustive search
   \end{itemize}
   \item Else:
   \begin{itemize}
      \item Repeat for $i=1$ to $\alpha$:
      \begin{itemize}
         \item Obtain $G'$ by performing $n-\lfloor n/\sqrt{\alpha}\rfloor$ random
contractions
         \item Compute $C' =$ Recursive-Randomized-Min-Cut$(G',\alpha)$
         \item If $i=1$ or $|C'|<|C|$, update $C \leftarrow C'$
      \end{itemize}
   \end{itemize}
   \item Return $C$
\end{enumerate}
```

```latex
\section{Deterministic vs Randomized Min-Cut}

\subsection{Deterministic Min-Cut}
\begin{itemize}
   \item Guarantees exact minimum cut
   \item Higher complexity for large graphs
   \item Stoer--Wagner complexity: $O(VE + V^2 \log V)$
\end{itemize}

\subsection{Randomized Min-Cut}
\begin{itemize}
   \item Produces minimum cut with high probability
   \item Time complexity: $O(V^2)$
\end{itemize}

\section{Probability of Correct Min-Cut}
The randomized algorithm outputs a true minimum cut with probability at least:
\[
\frac{2}{n(n-1)}.
\]

\section{Python Simulation}
Students are instructed to download the provided Jupyter notebook from the course platform and execute the simulation corresponding to Lecture~10.

\end{document}
```

If you'd like, I can next generate:

- **Professor-style viva questions from this lecture**, or
- **A perfectly formatted PDF-ready final scribe**, or
- **A memory-optimized 2-page revision sheet** for exam prep.

Just tell me which one you want next.